

David 总板子_1

5.20更新

主要包含图论和数据结构的主要模板

David 总板子_1

图论

- Tarjan缩点
- 倍增LCA
- K边最短路 bellman-ford
- 普通树链剖分
- 线段树树链剖分
- 动态维护带权树直径/直径端点/中心
- 简单树直径
- 简单树重心
- Spfa判负环
- 二分图相关
 - 匈牙利二分图最大匹配
 - 带权二分图最优匹配 KM算法
- 网络流相关
 - Dinic最大流
 - EK最大流
 - Dinic最小割

数据结构

- 莫队
 - 普通莫队
 - 带修莫队
 - 回滚莫队
- 区间加/乘线段树
- 二维线段树
- 李超线段树
 - 离散化李超线段树
- 主席树
- 带权并查集

图论

Tarjan缩点

```
1 int mod=1e9+7;
2 int MAXN=5e5+5;
3 vector<vector<int>> gra(MAXN); //原图邻接表
4 vector<int> val(MAXN); //原图点权
5 vector<int> dfn(MAXN); //原图dfn序, 用于构建scc
6 vector<int> low(MAXN); //节点的scc根节点, 用于构建scc
7 stack<int> stk; //栈, 用于构建scc
8 vector<int> instk(MAXN); //节点是否入栈, 用于构建scc
9 vector<int> itscc(MAXN); //ind to scc 原图点角标转新图点角标
10 vector<int> sccval(MAXN); //新图点权
11 vector<vector<int>> sccgra(MAXN); //新图邻接表
12 int cur=1; //维护原图dfn值
13 int curscc=0; //新图大小
```

```

14
15 int n,m;
16 vector<pair<int,int>> edge(MAXN);
17 void tarjan(int ind){
18     dfn[ind]=cur;
19     low[ind]=cur;
20     cur++;
21     instk[ind]=1;
22     stk.push(ind);
23     for(int i:gra[ind]){
24         //无向图这里加 if(par[i]==ind)continue;
25         if(!dfn[i]){
26             tarjan(i);
27             low[ind]=min(low[ind],low[i]);
28         }else if(instk[i]){
29             low[ind]=min(low[ind],dfn[i]);
30         }
31     }
32
33     if(low[ind]==dfn[ind]){
34         int cval=0;
35         int ctar=0;
36         int c;
37         do{
38             c=stk.top();
39             stk.pop();
40             instk[c]=0;
41             itscc[c]=curscc;
42             sccval[curscc]+=val[c];
43         }while(c!=ind);
44         curscc++;
45     }
46 }
47
48 void build(){
49     //第一步求解scc
50     for(int i=1;i<=n;i++){
51         if(!dfn[i])
52             tarjan(i);
53     }
54     //第二步建新图
55     for(int i=0;i<m;i++){
56         int a=edge[i].first;
57         int b=edge[i].second;
58         if(itscc[a]!=itscc[b]){
59             sccgra[itscc[a]].push_back(itscc[b]);
60         }
61     }
62 }
63
64 void solve(){
65     cin>>n>>m;
66     for(int i=0;i<m;i++){
67         int a,b;
68         cin>>a>>b;
69         gra[a].push_back(b);
70         edge.push_back({a,b});
71     }
72     for(int i=1;i<=n;i++){
73         cin>>val[i];
74     }

```

```

75
76     build();
77 }
78

```

倍增LCA

```

1  const int N=5e5+5;
2  vector<vector<int>> gra(N);
3  vector<vector<int>> par(N,vector<int>(19)); //父节点们
4  vector<int> dep(N); //深度
5  int n,q,root;
6
7  //dfs求深度和第一个父亲
8  void dfs(int ind){
9      for(int i:gra[ind]){
10         if(i==par[ind][0]) continue;
11         par[i][0]=ind;
12         dep[i]=dep[ind]+1;
13         dfs(i);
14     }
15 }
16
17 void solve(){
18     cin>>n>>q>>root;
19
20     for(int i = 1; i < n; i ++){
21         int a,b;
22         cin>>a>>b;
23         gra[a].push_back(b);
24         gra[b].push_back(a);
25     }
26
27     dfs(root);
28
29     //关键步骤：求每一个节点的第2^k个父亲是谁，可以nlogn递推求出
30     for(int i=1;i<=19;i++){
31         for(int j=1;j<=n;j++){
32             if(par[j][i-1]==0) continue;
33             par[j][i]=par[par[j][i-1]][i-1];
34         }
35     }
36
37     //先把两个点深度变为一样的，再同时往上走，使用倍增进行优化
38     for(int i=0;i<q;i++){
39         int a,b;
40         cin>>a>>b;
41         if(dep[a]>dep[b]){swap(a,b);}
42
43         for(int i=19;i>=0;i--){
44             if(par[b][i]==0) continue;
45             if(dep[par[b][i]]>=dep[a]){
46                 b=par[b][i];
47             }
48         }
49         if(b==a){
50             int lca=a;
51             cout<<lca<<endl;

```

```

52         continue;
53     }
54
55     for(int i=19;i>=0;i--){
56         if(par[b][i]==0)continue;
57
58         if(par[b][i]!=par[a][i]){
59             b=par[b][i];
60             a=par[a][i];
61         }
62     }
63     int lca=par[a][0];
64     cout<<lca<<endl;
65 }
66
67 }

```

K边最短路 bellman-ford

```

1  const int N = 1e5 + 5;
2  const int mod = 80112002;
3  int n, m, s, k, t;
4  int x, y, z;
5  vector<pair<int, int>>g[N];
6  int dist[N];
7  int back[N];
8  bool st[N];
9  //bellman-ford算法，有边数限制为k的单源最短路
10 int bell()
11 {
12     memset(dist, 0x3f, sizeof dist);
13     dist[1] = 0;
14     for (int p = 0; p < k; p++)
15     {
16         memcpy(back, dist, sizeof dist);
17         for (int j = 1; j <= n; j++)
18         {
19             for (auto& t : g[j])
20             {
21                 int a = t.first, b = t.second;
22
23                 dist[a] = min(back[j] + b, dist[a]);
24             }
25         }
26     }
27     if (dist[n] > 0x3f3f3f3f3f3f3f3f / 2) return 0x3f3f3f3f3f3f3f3f;
28     else
29         return dist[n];
30 }
31
32 signed main() {
33     ios::sync_with_stdio(false);
34     cin.tie(0);
35     cout.tie(0);
36
37     cin >> n >> m >> k;
38     for (int i = 1; i <= m; i++)
39     {

```

```

40         cin >> x >> y >> z;
41         g[x].push_back({ y,z });
42     }
43     int res = bell();
44     if (res == 0x3f3f3f3f3f3f3f3f) cout << "impossible" << endl;
45     else
46         cout << res << endl;
47
48     return 0;
49 }

```

普通树链剖分

```

1 //树链剖分求lca
2 //O(nlogn + q(logn)^2)
3 const int N=5e5+5;
4 vector<vector<int>> gra(N);
5 vector<int> par(N,0);//父节点
6 vector<int> dep(N);//深度
7 vector<int> siz(N);//子树大小
8 vector<int> mxchd(N);//最大子节点
9
10 vector<int> top(N);//当前节点重链的最顶端节点
11 vector<int> dfn(N);//重链优先的dfs序
12 vector<int> rdfn(N);//dfs序的逆,即rdfn[dfn[x]]=x
13
14 int n,q,root;
15 int dfnind = 1;
16 //第一次dfs找齐每一个节点的父节点,深度,子树大小和最大子节点
17 void dfs1(int ind){
18
19     top[ind] = ind;
20
21     int csize = 1;
22     int mxcd = 0;
23     int mxsiz = 0;
24     for(int i : gra[ind]){
25         if(i == par[ind]) continue;
26         par[i] = ind;
27
28         dep[i] = dep[ind] + 1;
29
30         dfs1(i);
31
32         csize += siz[i];
33
34         if(siz[i] > mxsiz){
35             mxsiz = siz[i];
36             mxcd = i;
37         }
38     }
39     siz[ind] = csize;
40     mxchd[ind] = mxcd;
41     return;
42 }
43 }
44
45 //第二次dfs进行重链优先的dfs,找到每一个节点的top,并且标好dfn序

```

```

46 void dfs2(int ind,int tp){
47
48     dfn[ind] = dfnind;
49     rdfn[dfnind] = ind;
50     dfnind ++;
51
52     top[ind] = tp;
53
54     if(mxchd[ind] != 0)
55     dfs2(mxchd[ind],tp);
56
57     for(int i : gra[ind]){
58         if(i == par[ind])continue;
59         if(i == mxchd[ind])continue;
60         dfs2(i, i);
61     }
62
63     return;
64 }
65
66 void solve(){
67     cin>>n>>q>>root;
68
69     for(int i = 1;i < n;i ++){
70         int a,b;
71         cin>>a>>b;
72         gra[a].push_back(b);
73         gra[b].push_back(a);
74     }
75
76     dfs1(root);
77     dfs2(root,root);
78
79     //求lca的原理：先找到两个点dfn序最大的那个点
80     //如果他的top比另一个点还要大，那么往上走一条链，直到top小于等于另外一个节点
81     //另外一个节点也同理，直到两个节点在同一条链上，dfn序最小的节点即为lca
82     for(int i=0;i<q;i++){
83         int a,b;
84         cin>>a>>b;
85
86         if(dfn[a] > dfn[b])swap(a, b);
87         while(dfn[top[a]] < dfn[top[b]]){
88             b = par[top[b]];
89             while(dfn[top[b]] < dfn[top[a]]){
90                 a = par[top[a]];
91             }
92         }
93
94         int lca = rdfn[min(dfn[a],dfn[b])];
95         cout<<lca<<endl;
96     }
97
98 }

```

线段树树链剖分

```

1 //线段树(辅助树链剖分)
2 struct Seg{
3     int n;
4     vector<int> tr, lazy;
5
6     Seg(int n) : n(n), tr(n * 4 + 5), lazy(n * 4 + 5) {}
7
8     void build(int p, int l, int r, const vector<int>& a){
9         if(l == r){
10             tr[p] = a[l];
11             return;
12         }
13         int mid = l + r >> 1;
14         build(p << 1, l, mid, a);
15         build(p << 1 | 1, mid + 1, r, a);
16         pushup(p);
17     }
18
19     void clear(){
20         fill(tr.begin(), tr.end(), 0);
21         fill(lazy.begin(), lazy.end(), 0);
22     }
23
24     void pushup(int p){
25         tr[p] = tr[p << 1] + tr[p << 1 | 1];
26     }
27
28     void pushdown(int p, int l, int r){
29         if(lazy[p]){
30             int mid = l + r >> 1;
31             tr[p << 1] += (mid - l + 1) * lazy[p];
32             tr[p << 1 | 1] += (r - mid) * lazy[p];
33             lazy[p << 1] += lazy[p];
34             lazy[p << 1 | 1] += lazy[p];
35             lazy[p] = 0;
36         }
37     }
38
39     void update(int p, int l, int r, int ql, int qr, int val){
40         if(ql <= l && r <= qr){
41             tr[p] += (r - l + 1) * val;
42             lazy[p] += val;
43             return;
44         }
45         pushdown(p, l, r);
46         int mid = l + r >> 1;
47         if(ql <= mid) update(p << 1, l, mid, ql, qr, val);
48         if(mid < qr) update(p << 1 | 1, mid + 1, r, ql, qr, val);
49         pushup(p);
50     }
51
52     int ask(int p, int l, int r, int ql, int qr){
53         if(ql <= l && r <= qr){
54             return tr[p];
55         }
56         pushdown(p, l, r);
57         int mid = l + r >> 1;
58         int res = 0;
59         if(ql <= mid) res += ask(p << 1, l, mid, ql, qr);
60         if(mid < qr) res += ask(p << 1 | 1, mid + 1, r, ql, qr);
61         return res;

```

```

62     }
63
64 };
65
66 //树链剖分
67 struct HLD{
68     int n, timer;
69     vector<int> sz, dep, fa, son, top, dfn, rnk;
70     vector<vector<int>> adj;
71     Seg seg;
72
73     HLD(int n) : n(n), timer(0), sz(n + 1), dep(n + 1), fa(n + 1), son(n + 1),
74     top(n + 1), dfn(n + 1), rnk(n + 1), adj(n + 1), seg(n) {}
75
76     void addEdge(int u, int v){
77         adj[u].push_back(v);
78         adj[v].push_back(u);
79     }
80
81     //处理 sz, dep, fa, son
82     void dfs1(int u, int p, int d){
83         sz[u] = 1; fa[u] = p; dep[u] = d; son[u] = 0;
84         for(int v : adj[u]){
85             if(v == p) continue;
86             dfs1(v, u, d + 1);
87             sz[u] += sz[v];
88             if(sz[v] > sz[son[u]]) son[u] = v;
89         }
90     }
91
92     //处理 top, dfn
93     void dfs2(int u, int tp){
94         top[u] = tp; dfn[u] = ++timer; rnk[timer] = u;
95         if(!son[u]) return;
96         dfs2(son[u], tp);
97         for(int v : adj[u]){
98             if(v == fa[u] || v == son[u]) continue;
99             dfs2(v, v);
100         }
101     }
102
103     //建树
104     void build(const vector<int>& a, int root = 1){
105         dfs1(root, 0, 1);
106         dfs2(root, root);
107         vector<int> v(n + 1);
108         for(int i = 1; i <= n; i++){
109             v[i] = a[rnk[i]];
110         }
111         seg.build(1, 1, n, v);
112     }
113
114     // loglog复杂度太大 可以用树上差分优化
115     void updatePath(int u, int v, int val){
116         while(top[u] != top[v]){
117             if(dep[top[u]] < dep[top[v]]) swap(u, v);
118             seg.update(1, 1, n, dfn[top[u]], dfn[u], val);
119             u = fa[top[u]];
120         }
121         if(dep[u] > dep[v]) swap(u, v);
122         seg.update(1, 1, n, dfn[u], dfn[v], val);

```



```

122     }
123
124     //询问路径
125     int askPath(int u, int v){
126         int res = 0;
127         while(top[u] != top[v]){
128             if(dep[top[u]] < dep[top[v]]) swap(u, v);
129             res += seg.ask(1, 1, n, dfn[top[u]], dfn[u]);
130             u = fa[top[u]];
131         }
132         if(dep[u] > dep[v]) swap(u, v);
133         res += seg.ask(1, 1, n, dfn[u], dfn[v]);
134         return res;
135     }
136
137     //更新树
138     void updateTree(int u, int val){
139         seg.update(1, 1, n, dfn[u], dfn[u] + sz[u] - 1, val);
140     }
141
142     //询问子树
143     int askTree(int u){
144         return seg.ask(1, 1, n, dfn[u], dfn[u] + sz[u] - 1);
145     }
146
147     //获得两点LCA
148     int getLCA(int u, int v){
149         while(top[u] != top[v]){
150             if(dep[top[u]] < dep[top[v]]) swap(u, v);
151             u = fa[top[u]];
152         }
153         return dep[u] < dep[v] ? u : v;
154     }
155
156     //获取两点间距离（无边权）
157     int getDist(int u, int v){
158         return dep[u] + dep[v] - 2 * dep[getLCA(u, v)];
159     }
160 };

```

动态维护带权树直径/直径端点/中心

```

1  const int MAXN=2e5+5;
2  const int LOG=32;
3
4  //{to,val}
5  vector<vector<pair<int,int>>> gra(MAXN);
6
7  vector<vector<int>> par(MAXN,vector<int>(LOG));
8
9  vector<int> vis(MAXN);
10 vector<int> dep(MAXN);
11 vector<int> dis(MAXN);
12
13 //初始化每个节点的父节点，深度，根节点到该节点的边权和
14 void dfs(int ind){
15     for(auto [i,w]:gra[ind]){
16         if(vis[i])continue;
17
18         vis[i]=1;

```

```

19
20     par[i][0]=ind;
21     dep[i]=dep[ind]+1;
22     dis[i]=dis[ind]+w;
23
24     dfs(i);
25 }
26 }
27
28 //求两节点LCA
29 int lca(int a,int b){
30     if(dep[a]>dep[b])swap(a,b);
31
32     for(int i=LOG-1;i>=0;i--){
33         if(dep[par[b][i]]>=dep[a]){
34             b=par[b][i];
35         }
36     }
37
38     if(a==b)return a;
39
40     for(int i=LOG-1;i>=0;i--){
41         if(par[a][i]!=par[b][i]){
42             a=par[a][i];
43             b=par[b][i];
44         }
45     }
46
47     return par[a][0];
48 }
49
50 //求两节点距离
51 int dist(int a,int b){
52     int l=lca(a,b);
53     return dis[a]+dis[b]-2*dis[l];
54 }
55
56 //从x节点向根节点移动k步的终点
57 int jump(int x,int k){
58     for(int i=0;i<LOG;i++){
59         if(k&(1LL<<i)){
60             x=par[x][i];
61         }
62     }
63     return x;
64 }
65
66 //获取从u向v走距离len时最远到达的位置，若落在一条边上返回靠近u的那个端点
67 int u_to_v_len(int u,int v,int len){
68     int l=lca(u,v);
69     int du=dist(u,l);
70     if(len<=du){
71         int cur=u;
72         for(int i=LOG-1;i>=0;i--){
73             int p=par[cur][i];
74             if(p && dist(u,p)<=len){
75                 cur=p;
76             }
77         }
78         return cur;
79     }

```

```

80     int rem=dist(u,v)-len;
81     int cur=v;
82     for(int i=LOG-1;i>=0;i--){
83         int p=par[cur][i];
84         if(p && dist(v,p)<=rem){
85             cur=p;
86         }
87     }
88     if(dist(cur,v)<rem)cur=par[cur][0];
89     return cur;
90 }
91
92 //获得u,v两点间中点所在点，若落在一条边上则返回这条边两端点，p1靠近u，p2靠近v
93 pair<int,int> get_center(int u,int v){
94     int d=dist(u,v);
95     int half=d/2;
96     int p=u_to_v_len(u,v,half);
97     if(dist(u,p)*2==d){
98         return {p,p};
99     }
100
101     int l=lca(u,v);
102     int nxt;
103     if(lca(u,p)==p&&p!=l){
104         int cur=u;
105         for(int i=LOG-1;i>=0;i--){
106             int pp=par[cur][i];
107             if(pp && dep[pp]>dep[p]){
108                 cur=pp;
109             }
110         }
111         nxt=par[cur][0];
112     }else{
113         int cur=v;
114         for(int i=LOG-1;i>=0;i--){
115             int pp=par[cur][i];
116             if(pp && dep[pp]>dep[p]){
117                 cur=pp;
118             }
119         }
120         nxt=cur;
121     }
122     return {p,nxt};
123 }
124
125 void solve(){
126     int n;
127     cin>>n;
128     dep[0]=-1;
129
130     for(int i=1;i<n;i++){
131         int a,b,w;
132         cin>>a>>b>>w;
133         gra[a].push_back({b,w});
134         gra[b].push_back({a,w});
135     }
136
137     int root;
138     cin>>root;
139     vis[root]=1;
140     dep[root]=1;

```

```

141     dfs(root);
142
143     for(int i=1;i<LOG;i++){
144         for(int j=1;j<=n;j++){
145             if(par[j][i-1]){
146                 par[j][i]=par[par[j][i-1]][i-1];
147             }
148         }
149     }
150
151     queue<int> que;
152     que.push(root);
153     int u=root;//当前直径端点
154     int v=root;//当前直径端点
155     int diameter=0;//当前直径
156     vis.assign(n+1,0);
157     vis[root]=1;
158
159     //这里是从根节点开始bfs，每一步都维护对应的直径，直径端点和中心
160     //具体题目修改对应的内容
161     while(que.size()){
162         queue<int> que2;
163         while(que.size()){
164             int tp=que.front();
165             que.pop();
166             for(auto [i,w]:gra[tp]){
167                 if(vis[i])continue;
168                 vis[i]=1;
169                 int d1=dist(u,i);
170                 int d2=dist(v,i);
171                 if(d1>diameter&&d1>d2){
172                     diameter=d1;
173                     v=i;
174                 }
175                 if(d2>diameter){
176                     diameter=d2;
177                     u=i;
178                 }
179                 pair<int,int> cen=get_center(u,v);
180                 cout<<u<<" "<<v<<" "<<diameter<<" ";
181                 if(cen.first==cen.second){
182                     cout<<cen.first<<"\n";
183                 }else{
184                     cout<<cen.first<<" "<<cen.second<<"\n";
185                 }
186                 que2.push(i);
187             }
188         }
189         swap(que, que2);
190     }
191     return;
192 }

```

简单树直径

```

1  const int N = 200005;
2
3  int n;

```

```

4  vector<int> g[N];
5  int dist[N];
6
7  void dfs(int u, int fa) {
8      for (int v : g[u]) {
9          if (v == fa) continue;
10         dist[v] = dist[u] + 1;
11         dfs(v, u);
12     }
13 }
14
15 int get_far(int start) {
16     memset(dist, 0, sizeof dist);
17     dfs(start, 0);
18
19     int far = start;
20     for (int i = 1; i <= n; i++) {
21         if (dist[i] > dist[far]) {
22             far = i;
23         }
24     }
25     return far;
26 }
27
28 int main() {
29     cin >> n;
30
31     for (int i = 1; i < n; i++) {
32         int u, v;
33         cin >> u >> v;
34         g[u].push_back(v);
35         g[v].push_back(u);
36     }
37
38     int s = get_far(1);
39     int t = get_far(s);
40
41     cout << dist[t] << '\n';
42
43
44     return 0;
45 }
46

```

简单树重心

```

1  const int N = 200005;
2
3  int n;
4  vector<int> g[N];
5
6  int sz[N];
7  int best = INT_MAX;
8  vector<int> centroid;
9
10 void dfs(int u, int fa) {
11     sz[u] = 1;
12

```

```

13     int max_part = 0;
14
15     for (int v : g[u]) {
16         if (v == fa) continue;
17
18         dfs(v, u);
19
20         sz[u] += sz[v];
21         max_part = max(max_part, sz[v]);
22     }
23
24     max_part = max(max_part, n - sz[u]);
25
26     if (max_part < best) {
27         best = max_part;
28         centroid.clear();
29         centroid.push_back(u);
30     } else if (max_part == best) {
31         centroid.push_back(u);
32     }
33 }
34
35 int main() {
36     cin >> n;
37
38     for (int i = 1; i < n; i++) {
39         int u, v;
40         cin >> u >> v;
41         g[u].push_back(v);
42         g[v].push_back(u);
43     }
44
45     dfs(1, 0);
46
47     cout << "Centroid:";
48     for (int x : centroid) {
49         cout << ' ' << x;
50     }
51     cout << '\n';
52
53     cout << "Min max component size = " << best << '\n';
54
55     return 0;
56 }

```

Spfa判负环

```

1  const int N = 1e4 + 5;
2  int t;
3  int n, m, u, v, w;
4  vector<pair<int,int> > arr[N];
5  int st[N], dist[N], cnt[N];
6  queue<int> q;
7  //spfa判断负环
8  void spfa()
9  {

```

```

10     for(int i=1;i<=n;i++)
11     {
12         st[i]=1;
13         q.push(i);
14     }
15     while (!q.empty())
16     {
17         int tm = q.front();
18         q.pop();
19         st[tm] = 0;
20         for (int i = 0; i < arr[tm].size(); i++)
21         {
22             int v = arr[tm][i].first, dis = arr[tm][i].second;
23             if (dist[v] > dist[tm] + dis)
24             {
25                 dist[v] = dist[tm] + dis;
26                 if (!st[v])
27                 {
28                     cnt[v] = cnt[tm] + 1;
29                     if (cnt[v] >= n)
30                     {
31                         cout << "Yes" << endl;
32                         return;
33                     }
34                     st[v] = 1;
35                     q.push(v);
36                 }
37             }
38         }
39     }
40     cout << "No" << endl;
41 }
42
43 signed main() {
44     ios::sync_with_stdio(false);
45     cin.tie(0);
46     cout.tie(0);
47
48
49     cin >> n >> m;
50     for (int i = 1; i <= m; i++)
51     {
52         cin >> u >> v >> w;
53         arr[u].push_back({v,w});
54     }
55     spfa();
56
57
58     return 0;
59 }

```

二分图相关

匈牙利二分图最大匹配

```

1  const int MAXN=505;      //右部大小
2  vector<int> state;        //右部已被左部匹配的状态
3  vector<int> vis(MAXN);   //右部的vis数组，用来判环

```

```

4  vector<vector<int>> vec; //从左部向右部建的有向边图
5  int visnum=1;          //标记法记vis, 节省时间
6  int check(int ind){
7      for(int i:vec[ind]){
8          if(vis[i]<visnum){
9              vis[i]=visnum;
10             if(state[i]==0||check(state[i])){
11                 state[i]=ind;
12                 return 1;
13             }
14         }
15     }
16     return 0;
17 }
18
19 void solve()
20 {
21
22     int n1,n2,m;
23     vec.assign(n1+1,vector<int>());
24     state.assign(n2+1,0);
25     //此时左部和右部已经被区分过, 如果没有被区分过先进行染色dfs操作
26     cin>>n1>>n2>>m;
27     for(int i=1;i<=m;i++){
28         {
29             int u,v;
30             cin>>u>>v;
31             //因为只从一边找另一边, 所以只用存一个方向, 即只建左部指向右部的边
32             vec[u].push_back(v);
33         }
34
35         int ans=0;
36         for(int i=1;i<=n1;i++){
37             visnum++;
38             if(check(i))ans++;
39         }
40         cout<<ans<<"\n";
41     }
42 }

```

补充定义和定理:

- 最大匹配数: 最大匹配的匹配边的数目
- 最小点覆盖数: 选取最少的点, 使任意一条边至少有一个端点被选择
- 最大独立集: 选取最多的点, 使任意所选两点均不相连
- 最小路径覆盖数: 对于一个 DAG (有向无环图), 选取最少条路径, 使得每个顶点属于且仅属于一条路径。路径长可以为 0 (即单个点)。
- 数量关系:
 - 定理1: 最大匹配数 = 最小点覆盖数 (这是 Konig 定理)
 - 定理2: 最大独立集 = 顶点数 - 最大匹配数
 - 定理3: 最小路径覆盖数 = 顶点数 - 最大匹配数

带权二分图最优匹配 KM算法

```

1  const int inf=0x7fffffff;
2  const int MAXN=505;

```



```

3
4 int cnta,cntb; //左部数量，右部数量
5 int w[MAXN][MAXN]; //边权，从左到右
6
7 //以下套用板子不用更改
8 int visa[MAXN],visb[MAXN]; //访问标记
9 int la[MAXN],lb[MAXN]; //顶标
10 int match[MAXN],upd[MAXN]; //匹配点、更新
11 int visnum=0;
12 bool check(int x) { //匈牙利
13     visa[x]=visnum;
14     for(int y=1; y<=cnta; y++){
15         if(w[x][y]!=-inf){
16             if(visb[y]<visnum)
17                 if(la[x]+lb[y]==w[x][y]) {
18                     visb[y]=visnum;
19                     if(!match[y]||check(match[y])) {
20                         match[y]=x;
21                         return true;
22                     }
23                 } else upd[y]=min(upd[y],la[x]+lb[y]-w[x][y]);
24         }
25     }
26
27     return false;
28 }
29
30 int KM() { //KM板子
31     memset(match,0,sizeof(match));
32     for(int i=1; i<=cnta; i++) {
33         la[i]=-inf;
34         lb[i]=0;
35         for(int j=1; j<=cntb; j++){
36             la[i]=max(la[i],w[i][j]);
37         }
38     }
39     for(int i=1; i<=cnta; i++){
40         while(true) {
41             visnum++;
42             for(int j=1; j<=cntb; j++){
43                 upd[j]=inf;
44             }
45             if(check(i))break;
46             int delta=inf;
47             for(int j=1; j<=cntb; j++){
48                 if(visb[j]<visnum)delta=min(delta,upd[j]);
49             }
50             for(int j=1; j<=cntb; j++) {
51                 if(visa[j]==visnum)la[j]-=delta;
52                 if(visb[j]==visnum)lb[j]+=delta;
53             }
54         }
55     }
56     int ans=0;
57     for(int i=1; i<=cntb; i++){
58         ans+=w[match[i]][i];
59     }
60     return ans;
61 }
62
63 void solve(){

```

```

64     cin>>cnta;
65     cntb=cnta;
66     for(int i=1;i<=cnta;i++){
67         for(int j=1;j<=cntb;j++){
68             {
69                 int in;
70                 cin>>in;
71                 w[i][j]=in;
72             }
73         }
74     int det=KM();
75     /*
76         如果要求完美匹配的最小权，将边权取负数w[i][j]=-w[i][j]
77         最后答案输出-KM()即可
78     */
79     cout<<det<<"\n";
80 }

```

网络流相关

Dinic最大流

```

1  const int N=210;
2  const int M=10010;
3  const int MOD=998244353;
4
5  int idx=1;//从2 3开始配对
6  int h[N];
7  int d[N],cur[N];
8  int S,T;
9  int n,m;
10
11 struct edge {
12     int v,c,ne;
13 }e[M];
14
15 void add(int a,int b,int c) {
16     e[++idx]={b,c,h[a]};
17     h[a]=idx;
18 }
19 bool bfs() {
20     memset(d,0,sizeof d);
21     queue<int> q;
22     q.push(S);
23     d[S]=1;
24     while (!q.empty()) {
25         int u=q.front();
26         q.pop();
27         for (int i=h[u];i!=-1;i=e[i].ne) {
28             int v=e[i].v;
29             if (d[v]==0&&e[i].c) {
30                 d[v]=d[u]+1;
31                 q.push(v);
32                 if (v==T) return true;
33             }
34         }
35     }
36     return false;

```

```

37 }
38 int dfs(int u,int mf) { //多路增广
39     if (u==T) return mf;
40     int sum=0;
41     for (int i=cur[u];i!=-1;i=e[i].ne) {
42         cur[u]=i;//当前弧优化
43         int v=e[i].v;
44         if (d[v]==d[u]+1&&e[i].c) {
45             int f=dfs(v,min(mf,e[i].c));
46             e[i].c-=f;
47             e[i^1].c+=f;//更新残留网
48             sum+=f;//累加u的流出流量
49             mf-=f;//减少u的剩余容量
50             if (mf==0) break;//余量优化
51         }
52     }
53     if (sum==0) d[u]=0;//残枝优化
54     return sum;
55 }
56
57 // O ( n * n * m )
58 int dinic() {
59     int flow=0;
60     while (bfs()) {
61         memcpy(cur,h,sizeof h);
62         flow+=dfs(S,INF);
63     }
64     return flow;
65 }
66
67 void solve()
68 {
69     memset(h,-1,sizeof h);
70     cin>>n>>m>>S>>T;
71     for (int i=1;i<=m;i++) {
72         int u,v,w;
73         cin>>u>>v>>w;
74         add(u,v,w);
75         add(v,u,0);
76     }
77     cout<<dinic()<<endl;
78 }

```

EK最大流

```

1  const int N=210;
2  const int M=10010;
3  const int MOD=998244353;
4
5  int idx=1;//从2 3开始配对
6  int h[N];
7  int mf[N],pre[N];
8  int S,T;
9  int n,m;
10
11 struct edge {
12     int v,c,ne;
13 }e[M]; //注意边集要开两倍大小

```

```

14
15 void add(int a,int b,int c) {
16     e[++idx]={b,c,h[a]};
17     h[a]=idx;
18 }
19 bool bfs() {
20     memset(mf,0,sizeof mf);
21     queue<int> q;
22     q.push(S);
23     mf[S]=INF;
24     while (!q.empty()) {
25         int u=q.front();
26         q.pop();
27         for (int i=h[u];i!=-1;i=e[i].ne) {
28             int v=e[i].v;
29             if (mf[v]==0&&e[i].c) {
30                 mf[v]=min(mf[u],e[i].c);
31                 pre[v]=i;
32                 q.push(v);
33                 if (v==T) return true;
34             }
35         }
36     }
37     return false;
38 }
39 // O ( n * m * m )
40 int EK() {
41     int flow=0;
42     while (bfs()) {
43         int v=T;
44         while (v!=S) {
45             int i=pre[v];
46             e[i].c-=mf[T];
47             e[i^1].c+=mf[T];
48             v=e[i^1].v;
49         }
50         flow+=mf[T];
51     }
52     return flow;
53 }
54
55 void solve()
56 {
57     memset(h,-1,sizeof h);
58     cin>>n>>m>>S>>T;
59     for (int i=1;i<=m;i++) {
60         int u,v,w;
61         cin>>u>>v>>w;
62         add(u,v,w);
63         add(v,u,0);
64     }
65     cout<<EK()<<endl;
66 }

```

Dinic最小割

```

1  const int N=210;
2  const int M=10010;

```

```

3  const int MOD=998244353;
4
5  int idx=1;//从2 3开始配对
6  int h[N];
7  int d[N],cur[N];
8  int S,T;
9  int n,m;
10 bool vis[N];
11
12 struct edge {
13     int v,c,ne;
14 }e[M];
15
16 void add(int a,int b,int c) {
17     e[++idx]={b,c,h[a]};
18     h[a]=idx;
19 }
20 bool bfs() {
21     memset(d,0,sizeof d);
22     queue<int> q;
23     q.push(S);
24     d[S]=1;
25     while (!q.empty()) {
26         int u=q.front();
27         q.pop();
28         for (int i=h[u];i!=-1;i=e[i].ne) {
29             int v=e[i].v;
30             if (d[v]==0&&e[i].c) {
31                 d[v]=d[u]+1;
32                 q.push(v);
33                 if (v==T) return true;
34             }
35         }
36     }
37     return false;
38 }
39 int dfs(int u,int mf) { //多路增广
40     if (u==T) return mf;
41     int sum=0;
42     for (int i=cur[u];i!=-1;i=e[i].ne) {
43         cur[u]=i;//当前弧优化
44         int v=e[i].v;
45         if (d[v]==d[u]+1&&e[i].c) {
46             int f=dfs(v,min(mf,e[i].c));
47             e[i].c-=f;
48             e[i^1].c+=f;//更新残留网
49             sum+=f;//累加u的流出流量
50             mf-=f;//减少u的剩余容量
51             if (mf==0) break;//余量优化
52         }
53     }
54     if (sum==0) d[u]=0;//残枝优化
55     return sum;
56 }
57
58 // O ( n * n * m )
59 int dinic() {
60     int flow=0;
61     while (bfs()) {
62         memcpy(cur,h,sizeof h);
63         flow+=dfs(S,INF);

```

```

64     }
65     return flow;
66 }
67
68 void mincut(int u) {
69     vis[u]=1;
70     for (int i=h[u];i!=-1;i=e[i].ne) {
71         int v=e[i].v;
72         if (!vis[v]&&e[i].c) mincut(v);
73     }
74 }
75
76 void solve()
77 {
78     memset(h,-1,sizeof h);
79     cin>>n>>m>>S>>T;
80     vector<int> a(n+5),b(n+5);
81     int BASE=m+1; // BASE > m
82     for (int i=1;i<=m;i++) {
83         int w;
84         cin>>a[i]>>b[i]>>w;
85         add(a[i],b[i],w*BASE+1);
86         add(b[i],a[i],0);
87     }
88     //最小割最大流定理
89     int ans=dinic();
90     cout<<ans/BASE<<' '<<ans%BASE<<endl; // 最小割(最大流) 大小 最小割的边数
91     //最小割的划分
92     mincut(S);
93     for (int i=1;i<=n;i++) {
94         if (vis[i]) cout<<i<<' '; // S
95         cout<<endl;
96     }
97     for (int i=1;i<=n;i++) {
98         if (!vis[i]) cout<<i<<' '; // T
99         cout<<endl;
100     }
101 }

```

数据结构

莫队

普通莫队

```

1
2  const int maxn = 5e4;
3  int B;
4  int cur = 0;
5
6  struct Q{
7      int l, r, id;
8
9      bool operator<(const Q& b)const{
10         int pos1 = l / B, pos2 = b.l / B;
11         if(pos1 != pos2) return pos1 < pos2;
12         if(pos1 & 1) return r < b.r;

```

```

13         return r > b.r;
14     }
15 };
16
17 vector<int> cnt(maxn + 5);
18 vector<int> a(maxn + 5);
19 vector<Q> ask(maxn + 5);
20 vector<pair<int,int>> ans(maxn + 5);
21
22 int l = 1, r = 0;
23
24 void add(int pos){
25     int val = a[pos];
26     cur += cnt[val];
27     cnt[val]++;
28 }
29
30 void del(int pos){
31     int val = a[pos];
32     cnt[val]--;
33     cur -= cnt[val];
34 }
35
36 void solve(){
37     int n, m;
38     cin >> n >> m;
39     B = max(1.0, n / sqrt(m * 2.0 / 3.0));
40     for(int i = 1; i <= n; i++){
41         cin >> a[i];
42     }
43     for(int i = 0; i < m; i++){
44         cin >> ask[i].l >> ask[i].r;
45         ask[i].id = i;
46     }
47     sort(ask.begin(), ask.begin() + m);
48     for(int i = 0; i < m; i++){
49         auto[q1, qr, id] = ask[i];
50         while(l > q1) add(--l);
51         while(r < qr) add(++r);
52         while(l < q1) del(l++);
53         while(r > qr) del(r--);
54         int len = r - l + 1;
55         int fm = cur == 0 ? 1 : len * (len - 1) / 2;
56         int g = cur == 0 ? 1 : __gcd(cur, fm);
57         ans[id] = {cur / g, fm / g};
58     }
59     for(int i = 0; i < m; i++){
60         cout << ans[i].first << '/' << ans[i].second << '\n';
61     }
62
63
64 }
65
66

```

```

2  int n, m, B;
3  struct Q{
4      int l, r, t, id;
5
6      bool operator<(const Q& b)const{
7          if(l / B != b.l / B) return l / B < b.l / B;
8          if(r / B != b.r / B) return r / B < b.r / B;
9          return t < b.t;
10     }
11
12     Q(int l, int r, int t, int id) : l(l), r(r), t(t), id(id) {}
13 };
14
15 struct OP{
16     int p, x;
17
18     OP(int p, int x): p(p), x(x) {}
19 };
20
21 vector<int> a;
22 vector<Q> ask;
23 vector<OP> op;
24 vector<int> cnt(1000006);
25 int res = 0;
26
27 void add(int val){
28     if(cnt[val] == 0) res++;
29     cnt[val]++;
30 }
31
32 void del(int val){
33     cnt[val]--;
34     if(cnt[val] == 0) res--;
35 }
36
37 void solve(){
38     cin >> n >> m;
39     a.resize(n);
40     for(int i = 0; i < n; i++){
41         cin >> a[i];
42     }
43
44     for(int i = 0; i < m; i++){
45         char c;
46         int x, y;
47         cin >> c >> x >> y;
48         if(c == 'Q'){
49             ask.push_back(Q(x - 1, y - 1, op.size(), ask.size()));
50         } else {
51             op.push_back(OP(x - 1, y));
52         }
53     }
54     B = pow(n, 2.0 / 3.0);
55     sort(ask.begin(), ask.end());
56     vector<int> ans(ask.size());
57     int curT = 0, l = 0, r = -1;
58     for(int i = 0; i < ask.size(); i++){
59         auto[q1, qr, t, id] = ask[i];
60         while(l > q1) add(a[--l]);
61         while(r < qr) add(a[++r]);
62         while(l < q1) del(a[l++]);

```



```

63     while(r > qr) del(a[r--]);
64     while(curT < t){
65         int pos = op[curT].p;
66         if(l <= pos && pos <= r){
67             del(a[pos]), add(op[curT].x);
68         }
69         swap(a[pos], op[curT].x);
70         curT++;
71     }
72     while(curT > t){
73         curT--;
74         int pos = op[curT].p;
75         if(l <= pos && pos <= r){
76             del(a[pos]), add(op[curT].x);
77         }
78         swap(a[pos], op[curT].x);
79     }
80     ans[id] = res;
81 }
82 for(int i : ans) cout << i << '\n';
83
84 }

```

回滚莫队

```

1
2  int n, q, B;
3  struct Q{
4      int l, r, id;
5
6      bool operator<(const Q& b) const{
7          if(l / B != b.l / B) return l / B < b.l / B;
8          return r < b.r;
9      }
10     Q(int l, int r, int id) : l(l), r(r), id(id) {}
11 };
12
13 vector<int> a;
14 vector<Q> ask;
15 vector<int> fst, lst;
16 vector<int> cnt(200005);
17
18 void solve(){
19     cin >> n;
20     a.resize(n);
21     fst.resize(n + 5, -1);
22     lst.resize(n + 5, -1);
23     for(int i = 0; i < n; i++){
24         cin >> a[i];
25     }
26     cin >> q;
27     B = max(1.0, n / sqrt(q * 2.0 / 3.0));
28     auto b = a;
29     sort(b.begin(), b.end());
30     b.erase(unique(b.begin(), b.end()), b.end());
31     for(int i = 0; i < n; i++){
32         a[i] = lower_bound(b.begin(), b.end(), a[i]) - b.begin();
33     }

```

```

34     for(int i = 0; i < q; i++){
35         int x, y;
36         cin >> x >> y;
37         ask.push_back(Q(x - 1, y - 1, i));
38     }
39     sort(ask.begin(), ask.end());
40     vector<int> ans(q);
41     int lb = -1, r = -1, res = 0;
42     for(int i = 0; i < q; i++){
43         auto[q1, qr, id] = ask[i];
44         int cur = q1 / B;
45
46         if(cur != lb){
47             fill(fst.begin(), fst.end(), -1);
48             fill(lst.begin(), lst.end(), -1);
49             res = 0;
50             lb = cur;
51             r = (cur + 1) * B - 1;
52         }
53
54         if(cur == qr / B){
55             int temp = 0;
56             vector<int> wait;
57             for(int j = q1; j <= qr; j++){
58                 int val = a[j];
59                 if(fst[val] == -1){
60                     fst[val] = j;
61                     wait.push_back(val);
62                 }
63                 lst[val] = j;
64                 temp = max(temp, j - fst[val]);
65             }
66             ans[id] = temp;
67             for(int val : wait){
68                 fst[val] = -1;
69                 lst[val] = -1;
70             }
71             continue;
72         }
73
74
75         while(r < qr){
76             r++;
77             int val = a[r];
78             if(fst[val] == -1) fst[val] = r;
79             lst[val] = r;
80             res = max(res, lst[val] - fst[val]);
81         }
82
83         int l = min(n, (cur + 1) * B);
84         int tmp = res;
85         stack<array<int,3>> wait; // val fst lst
86         while(l > q1){
87             l--;
88             int val = a[l];
89             wait.push({val, fst[val], lst[val]});
90             if(lst[val] == -1) lst[val] = l;
91             fst[val] = l;
92             res = max(res, lst[val] - fst[val]);
93         }
94

```

```

95     ans[id] = res;
96     res = tmp;
97
98     while(wait.size()){
99         auto[val, l, r] = wait.top();
100        wait.pop();
101        fst[val] = l;
102        lst[val] = r;
103    }
104
105 }
106
107 for(int i : ans) cout << i << '\n';
108
109 }

```

区间加/乘线段树

```

1  //线段树
2  struct Seg{
3      int n, mod;
4      vector<int> tr, lazyAdd, lazyMul;
5
6      Seg(int n, int mod) : n(n), mod(mod),
7      tr(n * 4 + 5),
8      lazyAdd(n * 4 + 5),
9      lazyMul(n * 4 + 5, 1) {}
10
11     void build(int p, int l, int r, const vector<int>& a){
12         if(l == r){
13             tr[p] = a[l] % mod;
14             return;
15         }
16         int mid = l + r >> 1;
17         build(p << 1, l, mid, a);
18         build(p << 1 | 1, mid + 1, r, a);
19         pushup(p);
20     }
21
22     void clear(){
23         fill(tr.begin(), tr.end(), 0);
24         fill(lazyAdd.begin(), lazyAdd.end(), 0);
25         fill(lazyMul.begin(), lazyMul.end(), 1);
26     }
27
28     void pushup(int p){
29         tr[p] = (tr[p << 1] + tr[p << 1 | 1]) % mod;
30     }
31
32     void apply(int p, int l, int r, int mul, int add){
33         tr[p] = (1LL * tr[p] * mul % mod + 1LL * (r - l + 1) * add % mod) % mod;
34         lazyMul[p] = 1LL * lazyMul[p] * mul % mod;
35         lazyAdd[p] = (1LL * lazyAdd[p] * mul % mod + add) % mod;
36     }
37
38     void pushdown(int p, int l, int r){
39         if(lazyMul[p] != 1 || lazyAdd[p] != 0){
40             int mid = l + r >> 1;

```

```

41         apply(p << 1, l, mid, lazyMul[p], lazyAdd[p]);
42         apply(p << 1 | 1, mid + 1, r, lazyMul[p], lazyAdd[p]);
43         lazyMul[p] = 1;
44         lazyAdd[p] = 0;
45     }
46 }
47
48 void updateMul(int p, int l, int r, int ql, int qr, int val){
49     if(ql <= l && r <= qr){
50         apply(p, l, r, val, 0);
51         return;
52     }
53     pushdown(p, l, r);
54     int mid = l + r >> 1;
55     if(ql <= mid) updateMul(p << 1, l, mid, ql, qr, val);
56     if(mid < qr) updateMul(p << 1 | 1, mid + 1, r, ql, qr, val);
57     pushup(p);
58 }
59
60 void updateAdd(int p, int l, int r, int ql, int qr, int val){
61     if(ql <= l && r <= qr){
62         apply(p, l, r, 1, val);
63         return;
64     }
65     pushdown(p, l, r);
66     int mid = l + r >> 1;
67     if(ql <= mid) updateAdd(p << 1, l, mid, ql, qr, val);
68     if(mid < qr) updateAdd(p << 1 | 1, mid + 1, r, ql, qr, val);
69     pushup(p);
70 }
71
72 int ask(int p, int l, int r, int ql, int qr){
73     if(ql <= l && r <= qr){
74         return tr[p];
75     }
76     pushdown(p, l, r);
77     int mid = l + r >> 1;
78     int res = 0;
79     if(ql <= mid) res = (res + ask(p << 1, l, mid, ql, qr)) % mod;
80     if(mid < qr) res = (res + ask(p << 1 | 1, mid + 1, r, ql, qr)) % mod;
81     return res;
82 }
83 };

```

二维线段树

```

1 // 二维线段树
2 struct Seg2D {
3     int n, m;
4     vector<vector<int>> tr, lazy;
5
6     Seg2D(int n, int m) : n(n), m(m) {
7         tr.assign(n * 4 + 5, vector<int>(m * 4 + 5, 0));
8         lazy.assign(n * 4 + 5, vector<int>(m * 4 + 5, 0));
9     }
10
11     void build(int p, int l, int r, const vector<vector<int>>& a) {
12         build_y(p, l, 1, m, l, r, a);

```

```

13     }
14
15     // 构建Y轴
16     void build_y(int px, int py, int ly, int ry, int lx, int rx, const
vector<vector<int>>& a) {
17         if(ly == ry) {
18             if(lx == rx) {
19                 tr[px][py] = a[lx][ly];
20             } else {
21                 tr[px][py] = tr[px << 1][py] + tr[px << 1 | 1][py];
22             }
23             return;
24         }
25         int mid = ly + ry >> 1;
26         build_y(px, py << 1, ly, mid, lx, rx, a);
27         build_y(px, py << 1 | 1, mid + 1, ry, lx, rx, a);
28         pushup_y(px, py);
29     }
30
31     void build(int px, int lx, int rx, const vector<vector<int>>& a) {
32         if(lx == rx) {
33             build_y(px, 1, 1, m, lx, rx, a);
34             return;
35         }
36         int mid = lx + rx >> 1;
37         build(px << 1, lx, mid, a);
38         build(px << 1 | 1, mid + 1, rx, a);
39         build_y(px, 1, 1, m, lx, rx, a);
40     }
41
42     void clear() {
43         for(auto& row : tr) fill(row.begin(), row.end(), 0);
44         for(auto& row : lazy) fill(row.begin(), row.end(), 0);
45     }
46
47     void pushup_y(int px, int py) {
48         tr[px][py] = tr[px][py << 1] + tr[px][py << 1 | 1];
49     }
50
51     void pushdown_y(int px, int py, int ly, int ry) {
52         if(lazy[px][py]) {
53             int mid = ly + ry >> 1;
54             tr[px][py << 1] += (mid - ly + 1) * lazy[px][py];
55             tr[px][py << 1 | 1] += (ry - mid) * lazy[px][py];
56             lazy[px][py << 1] += lazy[px][py];
57             lazy[px][py << 1 | 1] += lazy[px][py];
58             lazy[px][py] = 0;
59         }
60     }
61
62     // 内层Y轴更新
63     void update_y(int px, int py, int ly, int ry, int qly, int qry, int val) {
64         if(qly <= ly && ry <= qry) {
65             tr[px][py] += (ry - ly + 1) * val;
66             lazy[px][py] += val;
67             return;
68         }
69         pushdown_y(px, py, ly, ry);
70         int mid = ly + ry >> 1;
71         if(qly <= mid) update_y(px, py << 1, ly, mid, qly, qry, val);
72         if(mid < qry) update_y(px, py << 1 | 1, mid + 1, ry, qly, qry, val);

```

```

73     pushup_y(px, py);
74 }
75
76 // 外层x轴更新
77 void update(int p, int l, int r, int ql, int qr, int qly, int qry, int val) {
78     if(ql <= l && r <= qr) {
79         update_y(p, l, l, m, qly, qry, val);
80         return;
81     }
82     int mid = l + r >> 1;
83     if(ql <= mid) update(p << 1, l, mid, ql, qr, qly, qry, val);
84     if(mid < qr) update(p << 1 | 1, mid + 1, r, ql, qr, qly, qry, val);
85     // 更新当前节点
86     for(int py = 1; py <= m * 4; py++) {
87         tr[p][py] = tr[p << 1][py] + tr[p << 1 | 1][py];
88     }
89 }
90
91 // 内层Y轴查询
92 int ask_y(int px, int py, int ly, int ry, int qly, int qry) {
93     if(qly <= ly && ry <= qry) {
94         return tr[px][py];
95     }
96     pushdown_y(px, py, ly, ry);
97     int mid = ly + ry >> 1;
98     int res = 0;
99     if(qly <= mid) res += ask_y(px, py << 1, ly, mid, qly, qry);
100    if(mid < qry) res += ask_y(px, py << 1 | 1, mid + 1, ry, qly, qry);
101    return res;
102 }
103
104 // 外层X轴查询
105 int ask(int p, int l, int r, int ql, int qr, int qly, int qry) {
106     if(ql <= l && r <= qr) {
107         return ask_y(p, l, l, m, qly, qry);
108     }
109     int mid = l + r >> 1;
110     int res = 0;
111     if(ql <= mid) res += ask(p << 1, l, mid, ql, qr, qly, qry);
112     if(mid < qr) res += ask(p << 1 | 1, mid + 1, r, ql, qr, qly, qry);
113     return res;
114 }
115 };
116
117 // 使用示例
118 // int main() {
119 //     int n = 5, m = 5;
120 //     vector<vector<int>> a(n + 1, vector<int>(m + 1));
121 //
122 //     Seg2D seg(n, m);
123 //     seg.build(1, 1, n, a);
124 //
125 //     // 矩形区域 (x1,y1) 到 (x2,y2) 加 val
126 //     seg.update(1, 1, n, x1, x2, y1, y2, val);
127 //
128 //     // 查询矩形区域 (x1,y1) 到 (x2,y2) 的和
129 //     int sum = seg.ask(1, 1, n, x1, x2, y1, y2);
130 //
131 //     return 0;
132 // }

```

李超线段树

```
1 //普通李超线段树
2 //用于查询若干线段或者直线在某点处的最值
3 const double eps = 1e-9, INF = 1e18;
4
5
6 int sgn(double x){
7     if(fabs(x) < eps) return 0;
8     return x < 0 ? -1 : 1;
9 }
10
11 struct Line{
12     double k, b;
13     bool exists;
14     //更改处 (-INF)
15     Line(double k = 0, double b = INF) : k(k), b(b), exists(false) {}
16
17     double cal(int x)const{
18         return k * x + b;
19     }
20 };
21 //插入线段复杂度为 log * log 其余为 log (当前为维护最小值版本)
22 struct LCSeg{
23     int n;
24     vector<Line> tr;
25     LCSeg(int n) : n(n), tr(n * 4) {}
26
27     void add(int p, int l, int r, Line val){
28         if(!tr[p].exists){
29             tr[p] = val;
30             tr[p].exists = true;
31             return;
32         }
33         int mid = l + r >> 1;
34         //更改处 <
35         if(tr[p].cal(mid) > val.cal(mid)) swap(tr[p], val);
36         if(l == r) return;
37         //更改处 >
38         if(val.cal(l) < tr[p].cal(l)){
39             add(p << 1, l, mid, val);
40         } else {
41             add(p << 1 | 1, mid + 1, r, val);
42         }
43     }
44
45     // 插入一条全局直线
46     void addLine(double k, double b){
47         Line l(k, b);
48         l.exists = true;
49         add(1, 1, n, l);
50     }
51
52     // 插入一条在 [L, R] 范围内的线段
53     void addSeg(int p, int l, int r, int ql, int qr, Line val){
54         if(ql <= l && r <= qr){
55             add(p, l, r, val);
56             return;
57         }
```

```

57     }
58     int mid = l + r >> 1;
59     if(q1 <= mid) addSeg(p << 1, l, mid, q1, qr, val);
60     if(mid < qr) addSeg(p << 1 | 1, mid + 1, r, q1, qr, val);
61 }
62 // 查询 x 处的最大 y 值
63 double ask(int p, int l, int r, int x){
64     if(x < l || x > r || !tr[p].exists) return INF; //更改处-INF
65     double res = tr[p].cal(x);
66     if(l == r) return res;
67     int mid = l + r >> 1;
68     //更改处 max
69     if(x <= mid) return min(res, ask(p << 1, l, mid, x));
70     return min(res, ask(p << 1 | 1, mid + 1, r, x));
71 }
72 };
73

```

离散化李超线段树

```

1 //离散化李超线段树（对所有可能的x值进行离散化处理）（最大值版本）
2 const int INF = 2e18; // 维护最大值，初始b设为-INF
3
4 struct Line {
5     int k, b;
6     bool exists;
7     Line(int k = 0, int b = -INF) : k(k), b(b), exists(false) {}
8     int cal(int x) const {
9         return k * x + b;
10    }
11 };
12
13 struct LCSeg {
14     int n;
15     vector<Line> tr;
16     vector<int>& coords; //离散化后的数组，sort+erase(unique)
17
18     LCSeg(int n, vector<int>& c) : n(n), tr(n * 4), coords(c) {}
19
20     void add(int p, int l, int r, Line val) {
21         if (!tr[p].exists) {
22             tr[p] = val; tr[p].exists = true;
23             return;
24         }
25
26         int mid = l + r >> 1;
27
28         int mid_x = coords[mid - 1];
29         int l_x = coords[l - 1];
30         int r_x = coords[r - 1];
31
32         if (tr[p].cal(mid_x) < val.cal(mid_x)) swap(tr[p], val);
33         if (l == r) return;
34
35         if (val.cal(l_x) > tr[p].cal(l_x)) add(p << 1, l, mid, val);
36         else if (val.cal(r_x) > tr[p].cal(r_x)) add(p << 1 | 1, mid + 1, r, val);
37     }
38
39     void addLine(int k, int b) {
40         Line l(k, b);

```



```

41         l.exists = true;
42         add(1, 1, n, 1);
43     }
44
45     int ask(int p, int l, int r, int idx) {
46         if (!tr[p].exists) return -INF;
47         int cur_x = coords[idx - 1];
48         int res = tr[p].cal(cur_x);
49         if (l == r) return res;
50
51         int mid = l + r >> 1;
52         if (idx <= mid) return max(res, ask(p << 1, l, mid, idx));
53         return max(res, ask(p << 1 | 1, mid + 1, r, idx));
54     }
55 };
56
57 // auto get_id = [&](int x) {
58 //     return lower_bound(coords.begin(), coords.end(), x) - coords.begin() + 1;
59 // };
60
61 // for (int i = 1; i <= n; i++) {
62 //     int x_idx = get_id(pre[i]);    !!! 要引用离散化后的角标 (1-based)
63 //     dp[i] = seg.ask(1, 1, m, x_idx) + a * pre[i] * pre[i] + b * pre[i];
64 //     int new_k = -2 * a * pre[i];
65 //     int new_b = dp[i] + a * pre[i] * pre[i] - b * pre[i] + c;
66 //     seg.addLine(new_k, new_b);
67 // }

```

主席树

```

1 //主席树单点修改单点查询
2
3 const int N=1e6+5;
4
5 struct node{
6     int l,r;
7     ll val;
8 }tree[N*40];
9 //模版上写20, 但是我交了re, 写大一点过了, 有些玄学
10
11 vector<ll> vec(N);
12 vector<int> root(N); //root表示第i个版本的根节点的index
13
14 int top=0;
15 //在更新操作时进行动态开点的函数
16 int clone(int ind){
17     top++;
18     tree[top]=tree[ind];
19     return top;
20 }
21
22 //建树
23 int build(int ind,int l,int r){
24     top++;
25     ind=top;
26
27     if(l==r){
28         tree[ind].val=vec[l];

```

```

29     return ind;
30 }
31
32 int mid=(l+r)/2;
33 //实际上更新点可以有多种方法，我使用的方式是为了统一更新建树和查询函数参数的格式
34 //build里面的第一个参数意义不大，为了统一参数那么写的，写一样的就行
35 tree[ind].l = build(tree[ind].l,l,mid);
36 tree[ind].r = build(tree[ind].r,mid+1,r);
37 return ind;
38 }
39
40 //更新
41 int update(int ind,int l,int r,int t,ll v){
42     ind=clone(ind);
43
44     if(l==r){
45         tree[ind].val=v;
46         return ind;
47     }
48     int mid=(l+r)/2;
49     if(t<=mid){
50         tree[ind].l=update(tree[ind].l,l,mid,t,v);
51     }else{
52         tree[ind].r=update(tree[ind].r,mid+1,r,t,v);
53     }
54     return ind;
55 }
56
57 //查询
58 ll query(int ind,int l,int r,int t){
59     if(l==r){
60         return tree[ind].val;
61     }
62
63     int mid=(l+r)/2;
64     if(t<=mid){
65         return query(tree[ind].l,l,mid,t);
66     }else{
67         return query(tree[ind].r,mid+1,r,t);
68     }
69 }
70
71 int n,q;
72 void solve(){
73     cin>>n>>q;
74
75     for(int i = 1;i <= n;i ++){
76         cin>>vec[i];
77     }
78
79     root[0]=build(1,1,n);
80     for(int i=1;i<=q;i++){
81         int version;
82         int command;
83         cin>>version>>command;
84         if(command==1){
85             int t;
86             ll v;
87             cin>>t>>v;
88             root[i]=update(root[version],1,n,t,v);
89         }else{

```

```

90         int t;
91         cin>>t;
92         ll ans=query(root[version],1,n,t);
93         cout<<ans<<endl;
94         root[i]=root[version];
95     }
96
97 }
98
99 }
```

带权并查集

```

1  //带权并查集，维护每个连通块根节点到每个节点中某个权值关系
2  struct DSU{
3      int n;
4      vector<int> fa, sz, dis;
5
6      DSU(int n):n(n),fa(n+1),sz(n+1,1),dis(n+1,0){
7          for(int i=1;i<=n;i++){
8              fa[i]=i;
9          }
10     }
11
12     int find(int x){
13         if(fa[x]==x) return x;
14
15         int f=fa[x];
16         fa[x]=find(fa[x]);
17
18         dis[x]+=dis[f];
19
20         return fa[x];
21     }
22
23     bool merge(int x,int y,int d){
24         int fx=find(x);
25         int fy=find(y);
26
27         if(fx==fy){
28             return dis[y]-dis[x]==d;
29         }
30
31         if(sz[fx]>sz[fy]){
32             swap(x,y);
33             swap(fx,fy);
34             d=-d;
35         }
36
37         fa[fx]=fy;
38
39         dis[fx]=dis[y]-dis[x]-d;
40
41         sz[fy]+=sz[fx];
42
43         return true;

```

```
44     }  
45 };
```